

ATM

Coding Exercise

Two accounts, persisted locally,
covered by 143 tests.

ATM

Checking
\$949.50

Savings
\$900.35

OPERATIONS

Deposit

Withdraw

Transfer

From

Checking

To

Savings

Amount

\$ 0.00

Transfer

TRANSACTION HISTORY

Time	Type	Amount	Accounts	Resulting balance
Sep 22, 2026, 03:08:54 UTC	Transfer	\$300.75	Checking → Savings	Checking \$949.50 · Savings \$900.35
Sep 22, 2026, 03:08:54 UTC	Withdrawal	-\$400.40	Savings	\$599.60
Sep 22, 2026, 03:08:54 UTC	Deposit	+\$250.25	Checking	\$1,250.25

Hint: try the Konami code.

- Runs against a scratch SQLite database.
- The intentionally narrow scope keeps the review on correctness, architectural boundaries, and evidence.
- The Konami code is up, up, down, down, left, right, left, right, B, A.

Technology stack

FRONTEND

Razor Pages, vanilla JavaScript, CSS

No client framework or packages

BACKEND

C# 14 on .NET 10

ASP.NET Core

DATABASE

SQLite via EF Core 10

Code-first migrations

TESTING

xUnit, WebApplicationFactory, coverlet

- Minimal dependencies: no client-side framework or npm packages.
 - Pages are server-rendered Razor Pages with a small amount of vanilla JavaScript and one “hand-written” stylesheet.
- The solution is a modular monolith of four projects (Domain, Application, Infrastructure, Web) with dependencies pointing inward, recorded in [ADR-0002](#).
 - EF Core and SQLite live only in the Infrastructure project.
- Tests use xUnit, WebApplicationFactory for in-process HTTP, and coverlet for coverage.

Decisions and costs

DECISION	COST
Razor Pages	Full page reload per action
Modular monolith	One feature spans four projects
SQLite + EF Core	Single-writer ceiling
2-decimal USD	Single-currency support

Razor Pages ([ADR-0001](#))

- Cost: a full page reload per action.
- The server renders HTML with minimal JavaScript, so the whole app is written in C# and ships as a single unit. A separate JavaScript front end (e.g., a React single-page app) would add its own npm build and a JSON API for the browser to call, because the browser would fetch data instead of receiving finished pages.

Modular monolith ([ADR-0002](#))

- Cost: one feature spans four projects, so adding a transaction type touches Domain, Application, Infrastructure, and Web.
- The payoff is that business rules can be tested without the web host or the database. Microservices would add operational complexity with no payoff at this scope.

SQLite + EF Core ([ADR-0003](#))

- Cost: single-writer ceiling.
- A server database is the upgrade path if concurrent load ever matters. The next slide covers the datastore choice.

2-decimal USD ([ADR-0005](#))

- Cost: single-currency support.
- Amounts are decimal with at most two fractional digits, which rules out floating-point error. The two-digit rule fits USD but not many other currencies.

Datastore choice

In-memory would suffice.

CHOSEN

SQLite file

Survives restarts

CONSIDERED

SQLite in-memory

Lost on restart

Hand-written store

Owens atomicity

EF Core InMemory

No transactions

Redis

Extra process

Chosen: SQLite + EF Core

- An in-memory store would have been enough. I chose a persistent one so balances and history survive restarts ([ADR-0003](#)).
- SQLite – single-file DB with near-zero setup.
- EF Core – the ORM (the .NET counterpart to Hibernate).
 - Transactions: saves a transfer's debit, credit, and history rows in a single DB transaction, so they either commit or roll back as a unit.
 - Optimistic concurrency: rejects a write based on an outdated balance, e.g., two tabs withdrawing against the same \$1,000 and overdrawing it. Doesn't lock account rows; assumes conflicts are rare.

Considered

- Every option sits behind the same Application ports, so switching is an Infrastructure-only change.
- SQLite in-memory – an in-memory connection string with one connection held open. Keeps EF Core, migrations, transactions, and constraints, but data is lost on restart. The best in-memory choice, e.g., for a demo mode.
- Hand-written store – dictionaries behind a single lock. No dependencies, but the code itself must commit balance and history together instead of relying on a database transaction.
- EF Core InMemory provider – discouraged by Microsoft. It isn't relational, ignores transactions, and enforces no constraints, so it can't prove the invariants.
- Redis – a separate server process to install and run, and atomic transfers need MULTI/EXEC or Lua scripts. More operational machinery than this exercise needs, and still volatile unless its persistence is configured.

Best framework for the model

CHOSEN

ASP.NET Core

Compile-time checks but fewer examples

RUNNER-UP

Django

Most examples but runtime-only errors

Spring Boot

Typed but slow tests

Node.js

Popular but fast-changing

RANKED BY

How soon LLM mistakes surface

Ranked by how soon LLM mistakes surface

- A model writes, runs, and fixes code; the sooner an error appears, the fewer turns it wastes
- A fast-changing framework can confuse an LLM into mixing in a pattern from an older version; newer or niche frameworks, released after a model's training cutoff, are where it is weakest
- ASP.NET Core (chosen)
 - Pro: the compiler catches type and null errors before tests run
 - Pro: in-process HTTP tests run in seconds
 - Con: less public code; outdated patterns (Startup.cs) linger in examples
- Django
 - Pro: most public examples; stable API
 - Con: no static types, so errors wait for a test to hit them
- Spring Boot
 - Pro: huge corpus; static types
 - Con: wiring errors appear at startup; tests start slowly
 - Con: the javax-to-jakarta rename splits examples
- Node.js
 - Pro: huge corpus; TypeScript types
 - Con: no default stack; fastest-changing ecosystem

| How I AI'ed

FRAME

GPT-5.6 Sol

I approved spec, plan, ADRs

BUILD

Fable 5.1

Six phases, tests green per commit

Chose to minimize rework over tokens

REFINE

Opus 5.5, GPT-6 Sol

Docs, deck, fixes

Frame: GPT-5.6 Sol in Codex with my architect agent

- [Architetto](#) is my open-source architect agent: it bootstraps new repositories and records every foundational decision.
- Interviewed me on four open questions: time box, persistence, seed accounts, repo visibility.
- The model proposed the stack; I approved it.
- Scaffolded before any feature code: [SPEC.md](#), [PLAN.md](#), six [ADRs](#), [AGENTS.md](#).
 - [AGENTS.md](#) lists the rules for this repo: money is decimal, balances never go negative, etc.

Build: Fable 5.1 in Claude Code

- Why Fable 5.1: I know the best approach is to plan with the strongest model and execute with cheaper ones, but when I reached the implementation phase, I was running short on time. Therefore, I chose the strongest end-to-end model at the time (a whopping week ago, before Opus 5.5 shipped).
- Six phases in order: domain, application, persistence, web, HTTP tests and accessibility, deck.
- Each phase committed only with the build and every test green; domain tests written first.

Refine: Opus 5.5 and GPT-6 Sol in Codex

- Windows setup steps in [README.md](#), a Dependabot fix, the deck redesign, slide wording.
- The design direction came from [jony-vibe](#), my design-consultation agent; the first deck skipped that consultation and was redone.
- [c4po](#), my workspace-admin agent, moved the deck's visual checks from Keynote to PowerPoint after Keynote passed a deck that PowerPoint had to repair.

Roadmap

OMITTED

Identity

Banking breadth

Distributed operations

NOW

Idempotency keys, cross-model review, authentication

NEXT

Accessibility, observability

LATER

Deployment

Omitted (out of scope in [SPEC.md](#))

- Identity
 - No login, card, PIN, or session; one local user
- Banking breadth
 - No fees, interest, overdraft lines, scheduled payments, other currencies, or bank rails
- Distributed operations
 - One process, one SQLite file
 - In a real bank, Domain and Application would become a backend service shared by every client (ATMs, mobile app, online banking, branch tellers)

Roadmap

- Now
 - Idempotency keys: a retry after a lost response repeats the operation; Post/Redirect/Get and EF Core's concurrency token cannot catch it
 - Cross-model code review: Fable 5.1 wrote the implementation, so have a different model (GPT-6 Sol) review it; a second model does not share the author's blind spots
 - Authentication: prerequisite for multiple users and attributable audits
- Next
 - Accessibility: screen-reader testing, axe or Lighthouse in CI
 - Known gap: with JavaScript on, Tab cannot reach the Withdraw and Transfer tabs (in BACKLOG.md)
 - Observability: structured logs, metrics, traces
- Later
 - Deployment: container, CI/CD, hosting